

An Engine-independent Framework for Business Rules Development

Mukundan K. Agaram

*Enterprise Architect, Information Systems and Support
Delta Dental of Michigan
Okemos, USA
Email: magaram@deltadentalmi.com*

Chang Liu

*Lead Developer, Information Systems and Support
Delta Dental of Michigan
Okemos, USA
Email: cliu@deltadentalmi.com*

Abstract—There is a compelling need for highly customized Domain Specific Languages and Business Vocabulary in certain industries such as insurance, mortgage, and finance to enable Knowledge Workers to articulate and to automate complex rules pertinent to their areas of function within their companies. Rule Engine vendors attempt to provide a solution to the problem by selling an integrated Rules Engine and Business Rules Management System. Usually, the BRMS's provided by vendors need to be customized and integrated into the overall Enterprise Architecture. This results in the Enterprise Architecture to be tightly coupled with the vendor's rule offering. Moreover, it poses a significant risk to the Enterprise as vendor solutions change between releases. The Enterprise Architecture needs a way to insulate itself from such impacts. This paper describes a framework that delivers the core BRMS functions of authoring and representation in a vendor neutral fashion. In addition, the paper sheds light on specific areas of the framework that can be standardized.

Keywords-Business Rules; Business Rules Languages; Business Rule Components; Business Vocabulary; Domain Specific Languages;

I. INTRODUCTION

Certain industries such as insurance, mortgage, and finance are highly regulated and policy driven. The Business Applications that are deployed in such scenarios contain a large number of rules. Typically, Rules Engines are used to execute these rules. Vendors in this area bundle the Rules Engine with the Business Rules Management System (BRMS). Also, the Rule Engines provided by vendors require proprietary Production Technical Rule Language for the rules to be interpreted by the engine. In addition, the BRMS provided by the vendor comes with a non-technical albeit proprietary Production Rule Language that allow the users to articulate the rules with naturally structured language and vocabulary. These authored rules are then stored in a vendor-specific format. The problem arises when these rules have to be converted or upgraded to the next release or a different rule vendor solution. In the worst case scenario, no automated solution is available for the upgrade. This brings up a strong need to author and represent the rules in a vendor neutral manner.

The framework offered in this paper is a result of the experience and insights gained from an overall upgrade effort to our business rules infrastructure. Further this work

augments an overall business rule architecture and framework called BRIDE (Business Rules Interactive Development Environment) [1]. As shown in Figure 1, the original BRIDE framework is built on top of ILOG JRules, a leading commercial rule product, and uses the BRMS components and the rule engine provided by the product. In Figure 2, we can see that the proposed framework improves upon the original framework by decoupling the BRIDE architecture from the underlying rule vendor implementations.

The original BRIDE framework provides a lot of additional features to rule development which the rule vendor solution does not offer out of the box. Needless to say, a significant amount of customization and innovation has been invested into the framework. Furthermore, the framework is a corner stone for the overall enterprise architecture. However, due to the tight integration with the underlying rule vendor solution a shift in the vendor solution between releases imposes a huge cost and risk to the Enterprise Architecture. To help mitigate this risk from a long term perspective, the framework needed to be decoupled from the underlying rule engine vendor solution.

The framework facilitates collaboration between a number of different actors within the enterprise. Figure 3 shows the user cases. The vendor solution has a demarcation between vocabulary development and applying it to their proprietary rule repository. However, with the new framework, these two functions are seamlessly integrated so that the "Apply Vocabulary" use case is no longer needed. We now look at some drawbacks of the original BRIDE framework and how the proposed framework architecture addresses them.

A. Limitations of the original framework

The original framework stores data in two areas, namely the BRIDE rule repository and the ILOG JRules Rule repository. The latter is a proprietary file format that stores the authored rule representations. Ever so often there is a use case of whole scale refreshing of the test BRIDE environments from production. The process involves copying the production BRIDE rule repository database and the flat file repositories. Having to maintain two distinct data stores makes this process cumbersome and error prone. There is a strong requirement to be able to migrate rules from one

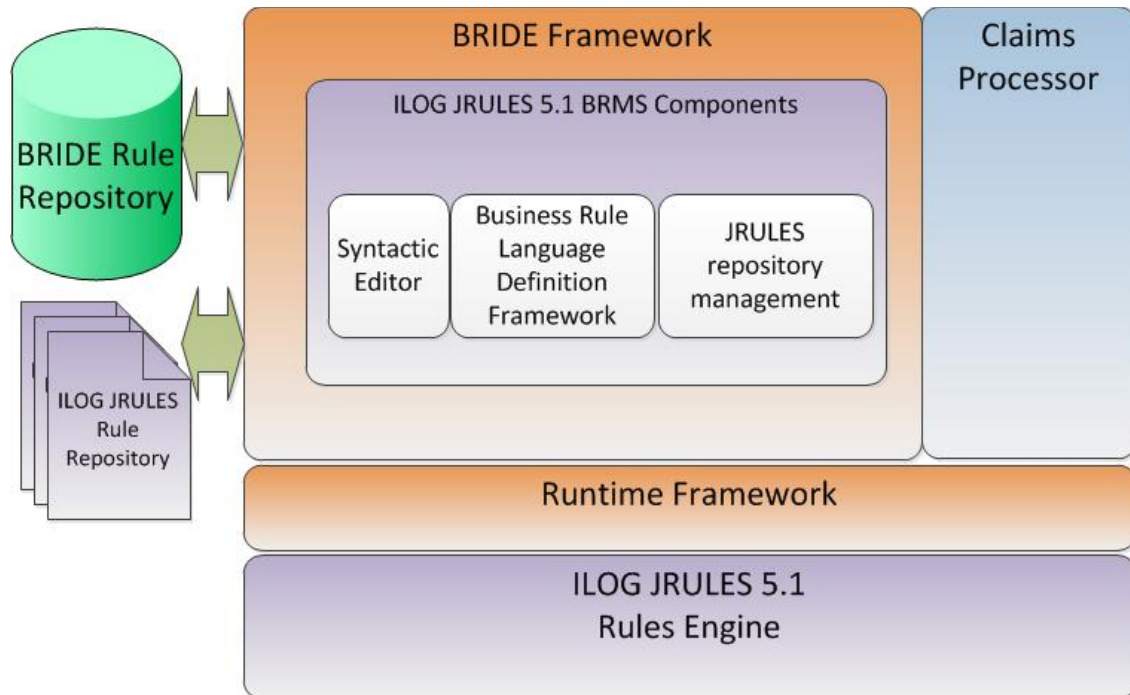


Figure 1. Original BRIDE Architecture

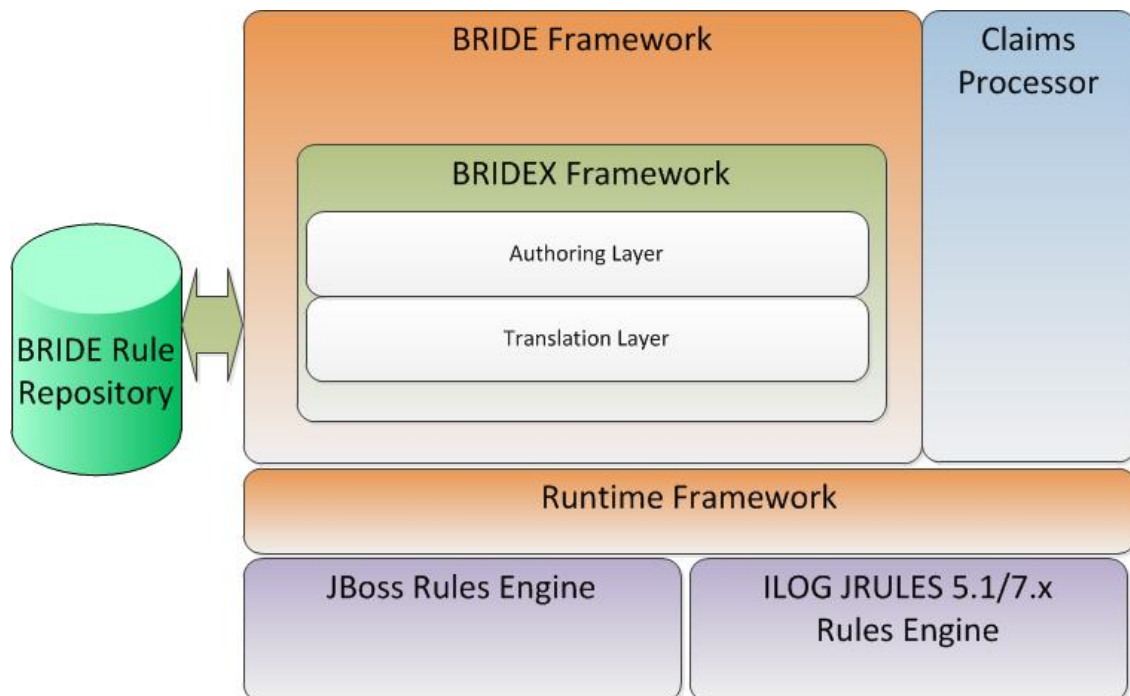


Figure 2. Proposed BRIDE Architecture

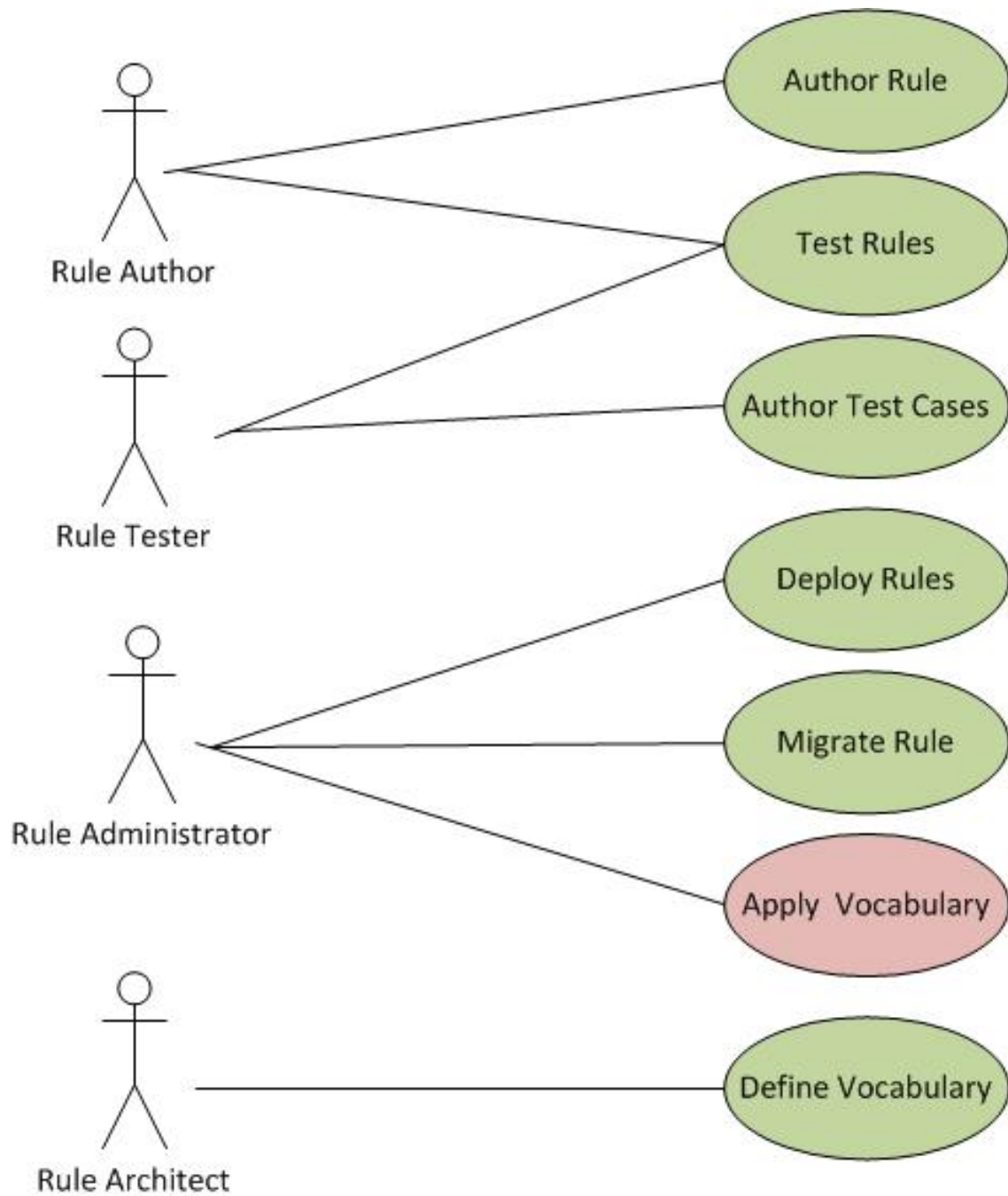


Figure 3. Predominant Rule Use Cases

environment to another environment. For example TEST to User Acceptance Test (UAT). There is a “Migrate Rule” feature in the original BRIDE framework that addresses this. Unfortunately, it can only partially resolve this issue. Due to the limitations of the vendor solution and the proprietary rule repository it is not possible to migrate a standalone rule from one environment to another. Only entire rule repositories can be migrated and the migration can only happen if the repository does not exist in the target environment. Furthermore, based on the rule Realization [1], the development of the rules can be originated from production or development. Therefore, there is a strong requirement of bidirectional migration of specific rules from one environment to another. To overcome this migration limitation, the rule authors are forced to re-author the same rules in multiple environments, which can be error prone. Lastly, some of the dental procedure centric rules take a pattern of dental procedures as part of the vocabulary. This pattern is free form text to enable fast rule entry. The vendor solution does not provide a way to hook in custom validation to the editor to catch user data entry errors.

B. Advantages of the proposed framework

The proposed framework stores data in a single storage, i.e. the BRIDE rule repository. Therefore, this alleviates the whole scale refresh issue going forward. Secondly, since the BRIDE rule repository is the single integrated data source and since the framework holds the knowledge for the rule representation, it is now possible to enhance the “Migrate Rule” use case’s capability of migrating standalone rules from one environment to another. This will remove the necessity for dual/redundant rule authoring in multiple environments. Lastly, since the framework defines its own Vocabulary Model and specification, it is possible to build in hook points to inject in custom validation to the editor that can catch user rule authoring data entry errors.

The business rules infrastructure addressed by this framework at this point is limited to the representation of Production Rule Systems. It helps to design a Business Rules Management System that is customized to the organization’s specific requirements without having to have a large footprint of customized vendor integration code.

The paper is organized as follows. In section II, we present the architecture of the framework and explain how the core components interact with each other. In section III, we elaborate the models that govern the integration of different components in the framework. In section IV, we explore the areas that can benefit from being standardized. V will summarize the paper, draw conclusions from this work and expand on potential future work based on the insights gained with this work.

II. FRAMEWORK

The framework can broadly be broken up into an Authoring Layer which sits upon a Translation Layer. The authoring layer is primarily responsible for the authoring of rules in a format understood by a non-technical business expert. The authoring layer then generates an Intermediate Rule Representation which is passed to the translation layer. The translation layer can then translate it to one or more target rule engines. A vocabulary model supports the development of a domain specific fact model that is used by both the authoring layer and the translation layer. The framework can be integrated into the landscape of a Business Rules Management System. Figure 4 depicts the high-level architecture of this framework. In this section, we will take a deep look at the different components that constitute this framework.

A. Authoring Layer

The authoring layer acts as the presentation layer. It contains a guided rule editor component that can be integrated with the Business Rules Management System. The guided rule editor is backed by the rule model that contains the language syntax elements and vocabulary elements required to author a rule. The elements of the rule model are generated by the language parsing unit which parses the language specification and the domain vocabulary specification. When a new rule needs to be authored by the user, the guided rule editor presents an empty rule model as a rule template that has the syntactic and vocabulary elements needed to author a rule. The user then uses the guided rule editor to click, guide and select the necessary syntax and vocabulary elements to articulate the rule.

The rule model can generate a serialized representation of the authored rule which is stored in the rule repository and can be loaded back into the editor by the BRMS. The rule model can also generate an intermediate rule representation which can then be translated to specific rule engine target languages. The language parsing unit has parsers that can parse the rule specification language and the vocabulary specification language. The vocabulary elements of the rule model are housed in the vocabulary model. This meta-data serves two major purposes. Firstly, they augment the rule model with the vocabulary specific elements needed for the authoring of a rule. Secondly, the meta-data is surfaced to the translation layer where the rule is translated to specific rule engine targets. Figure 5 shows the high level design of the authoring layer.

B. Translation Layer

The primary input for the translation layer is the intermediate rule representation. The translation layer is made up of an intermediate rule language parser that can parse the intermediate rule. The parser integrates with an abstract rule target generator that is responsible for generating the target

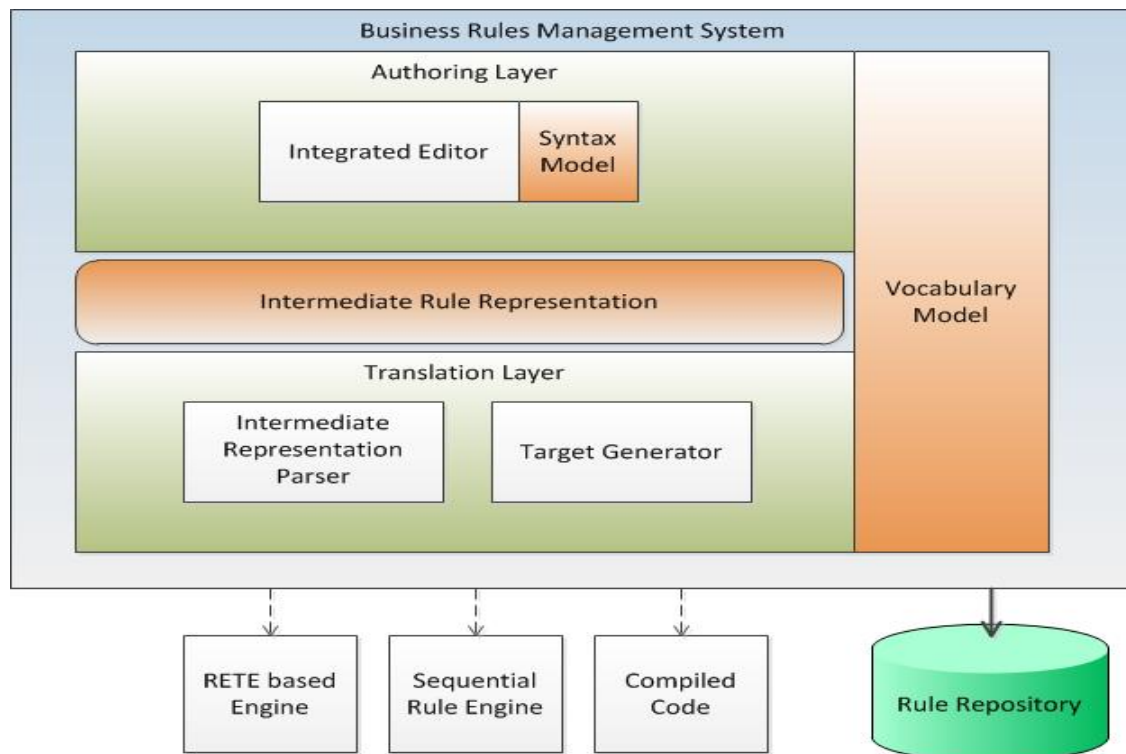


Figure 4. Framework Blueprint

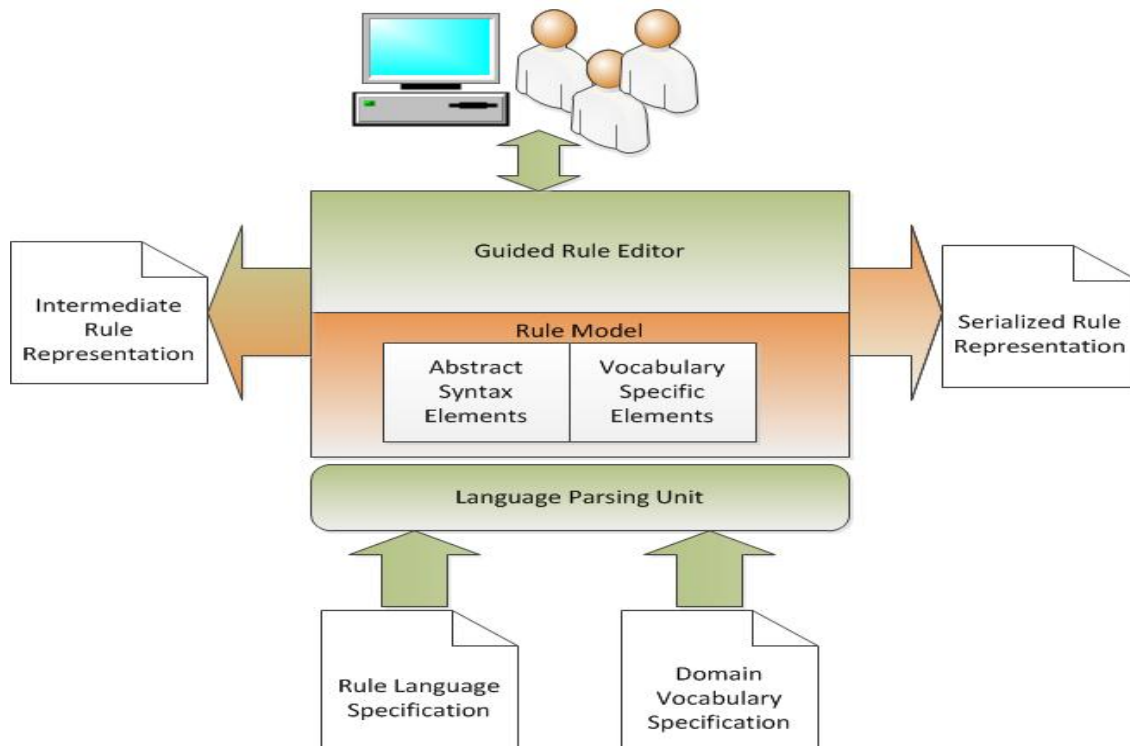


Figure 5. Authoring Layer

rule language. The abstract rule target generator is backed by one or more concrete rule target generator. Each concrete rule target generator is responsible for generating the rule in a target rule language. The target rule language could be for a RETE based rule engine or a non-RETE based rule engine. It could even be procedural code if the situation demands. For example, there can be an IRL Rule Target Generator for ILOG Rule Language (IRL), a JBoss Rule Target Generator for Drools Rule Language (DRL), and so on and so forth. It should be possible to even generate a procedural target like java byte code. The intermediate rule is represented in a language that incorporates the common syntactical elements and the vocabulary elements of the underlying business domain needed to express the rule. Care is taken in the intermediate rule representation to demarcate the vocabulary elements from the syntactical elements so that the intermediate rule language parser can delegate areas of vocabulary parsing to a specialized vocabulary parser. The parsers work closely with the generators who collect the different syntactic and vocabulary elements to eventually generate concrete target code that is executable. Figure 6 shows the high level design of the translation layer.

III. MODELS

As discussed in Section II, Syntax Model and Vocabulary Model provide guidance over the process from rule authoring to execution. They can be considered the blueprint that depicts the language structure of business rules. By specifying various syntax and vocabulary models, the framework provides the flexibility of defining domain specific rule languages that fit the business users' need while making them executable on different Rule Engines.

In this section, an example of History Cross Check rule language will be used to illustrate how syntax model and vocabulary model are used to guide the authoring/translation process. For simplicity, implementation details are omitted.

When processing incoming claims, the rules in History Cross Check (HCC) can be articulated as “if ... then ...” production rules. But for business rules authors, a more comprehensible format would be preferred to categorized conditions into two categories, “History Cross Check Criteria” and “Benefit Limitations”. Therefore, it is necessary to have a Domain Specific Language and corresponding vocabulary geared towards articulating such rules. Figure 7 is an example HCC rule.

In plain English, the rule can be described as following:
If the input claim procedure is an examination/evaluation and the dentist is either an Oral Surgeon, Endodontist, Orthodontist, Periodontist or a Prosthodontist and there are more than two occurrences (including the current claim) of examination/evaluation done by the same dentist or dental office in a 12 month period based on the input date of service, the input claim will be denied.

History Cross Check Criteria:

Input Procedure Code in : [D0120; D0145; D0150; D0160; D0180](#)

and History Procedure Code in : [D0120; D0145; D0150; D0160; D0180](#)

and Input Provider's Specialty Code in : [010-020; 040-050](#)

and History and Input have the same Provider

+

Benefit Limitations:

Exceeded : [2](#) +|-*/ in : [12](#) +|-*/ : MONTHS

+

Action to take:

Deny and charge the Approved Fee

Apply Processing Policy code : [153](#)

Stop HCC Processing

+

Figure 7. A History Cross Check (HCC) Rule

A. Syntax Model

A Syntax Model specifies the syntactic structure of the Domain Specific Rule Language (HCC language in this example) that is more friendly to business users compared to technical rule languages. By looking into the HCC rule language, we can figure out its syntax that can be described in Extended Backus-Naur-Form (EBNF) as follows:

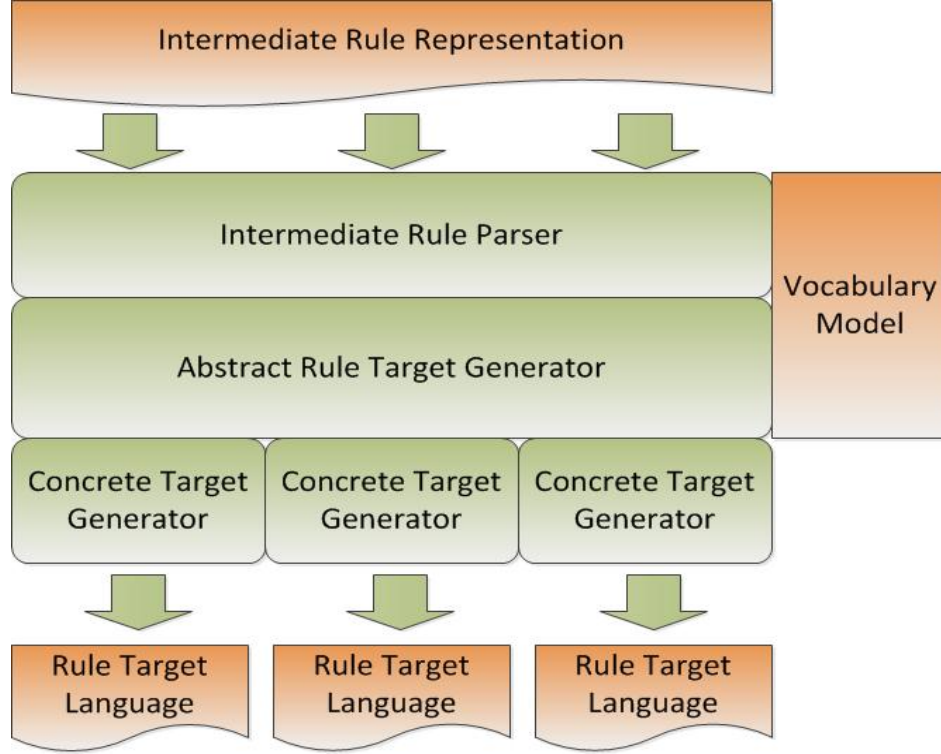


Figure 6. Translation Layer

```

rule = hccCriteria, benefitLimitations, decision;
hccCriteria = 'History Cross Check Criteria:', hccConditions;
benefitLimitations = 'Benefit Limitations:', blConditions;
decision = 'Actions to take:', actions;

```

```

hccConditions = hccCondition, {'and', hccCondition};
blConditions = blCondition, {'and', blCondition};
actions = {actions};

```

```

hccCondition = ? hcc vocabulary ?;
blCondition = ? benefit limitations vocabulary ?;
action = ? action vocabulary ?;

```

Notice that in this grammar, “hcc vocabulary”, “benefit limitations vocabulary”, “action vocabulary” are special sequence terminals whose contents will be determined according to the vocabulary model.

B. Vocabulary Model

The Syntax Model governs the syntactic structure of the rule language that is laid out to rule authors, while vocabulary details are yet to be completed. Consider the HCC rule shown in Figure 7. Being guided by its Syntax Model,

we create four HCC conditions. But this is how far Syntax Model can go. It is up to the Vocabulary Model to provide further guidance in populating these HCC conditions.

Moreover, to translate these natural language like business rules to a format that can be understood by a rule engine, it is necessary to map vocabulary to corresponding underlying implementation aspect such that the business rules execution ends up with expected runtime behavior. The Vocabulary Model is the bridge that fulfills this gap.

In the HCC rule language example, its Vocabulary Model mapping mechanism is shown in Table I. For instance, the Benefit Limitation, “Exceeded 2 in 12 MONTHS”, will translated to method invocation “frequencyLimitation(2, 12, DurationType.MONTH)”.

Although the Vocabulary Model mapping mechanism may seem straightforward in our HCC rule example, the specification of argument binding relationship should not be overlooked. Consider the vocabulary mapping shown in Figure 8. In order to accommodate a more general mapping like this, a Vocabulary Model needs to specify how parameters appear in Vocabulary (“ X_i ”, “ Y_i ”, “ $DATE_i$ ”) are bound to the arguments in the Implementation Aspect (“startDate”, “endDate”, “limit”). Therefore, in addition to the table data structure in Table I, argument binding artifacts are necessary to be introduced. In the scenario shown in Figure 8, the argument binding relationship includes not

Table I
VOCABULARY MODEL MAPPING TABLE

Vocabulary	Implementation Aspect	Description
Input Procedure Code in: $\langle procedureCodeRange \rangle$	boolean inputProcedureInRanges(String procedureCodeRange)	Validate input procedure code range.
History Procedure Code in: $\langle procedureCodeRange \rangle$	boolean historyProcedureInRanges(String procedureCodeRange)	Validate history procedure code range.
Input Provider's Specialty Code in: $\langle procedureCodeRange \rangle$	boolean inputSpecialtyInRanges(String procedureCodeRange)	Validate provider's specialty procedure code range.
History and Input have the same Provider	boolean sameProvider()	Whether the input and history claim have the same provider
Exceeded $\langle X \rangle$ in $\langle Y \rangle$ $\langle duration \rangle$	boolean frequencyLimitation(int X, int Y, DurationType duration)	Whether exceeds the benefit limitation
Deny and charge the Approved Fee	void denyApprovedFeeChargeable()	Deny the claim and charge the approved fee.
Apply Processing Policy code: $\langle policyCode \rangle$	void applyPolicy(String policyCode)	Apply a specific Processing Policy code.
Stop HCC Processing	void retractAll()	Stop HCC Processing.
...	...	...

only the positional binding (“ iX_i ” corresponds to the third argument “limit”, and “ $iDATE_i$ ” goes to the first argument “startDate”), but also more complicated semantic binding (the second argument “endDate” is the date of “ iY_i ” after “ $iDATE_i$ ”).

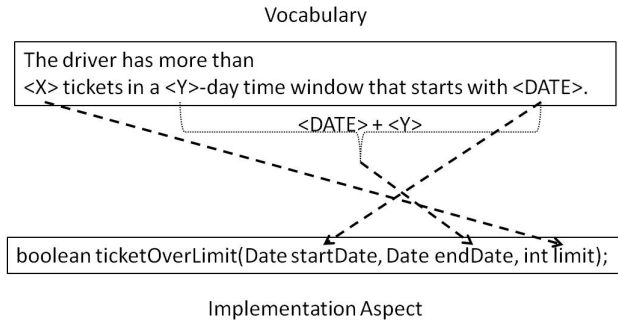


Figure 8. Argument Binding Relationship

The separation of Syntax Model and Vocabulary Model makes it possible for the business rule application developers to design a business user comprehensible rule representation for runtime behaviors while having the freedom of choosing different flavors of Implementation Aspects that would be required by the underlying rule engines. With this facility, the coupling between the Enterprise Architecture and a vendor-specific business rules solution is alleviated. Consider the situation where the architect of a business rule application would like to move away from a commercial rule engine to an open source one. All that needs to be done is to revise the Vocabulary Model mapping by targeting the translation to the format of the new rule engine.

C. Intermediate Representation

The Intermediate Representation is the output from the Authoring Layer and the input to the Translation Layer. In

order for the Translation Layer to do its job, the Authoring Layer should generate the Intermediate Representation in a format such that necessary rule meta data can be retrieved or inferred from.

For example, the Intermediate Representation for the business rule given in Figure 7 may look as follows:

```

If
  <vocab>inputProcedureInRanges
    ("D0120; D0145; D0150; D0160; D0180") </vocab>
  and
  <vocab>historyProcedureInRanges
    ("D0120; D0145; D0150; D0160; D0180") </vocab>
  and
  <vocab>inputSpecialtyInRanges
    ("010-020; 040-050") </vocab>
  and
  <vocab>sameProvider() </vocab>
  and
  <vocab>frequencyLimitation
    (2, 12, DurationType.MONTH) </vocab>

Then
  <vocab>denyApprovedFeeChargeable() </vocab>
  <vocab>applyPolicy("153") </vocab>
  <vocab>retractAll() </vocab>

```

Meta data in the Intermediate Representation should include (but not be limited to) information as follows,

- Which part of the representation is for rule conditions, and which part is for rule actions
- The starting and ending point of a vocabulary element (*</vocab>*, *</vocab>* tags serve for this purpose)

All technical production rule languages follow the structure of Left Hand Side (LHS) depicting rule conditions and Right Hand Side (RHS) describing rule actions. Meanwhile, the flexibility of defining Syntax/Vocabulary Model in the framework poses infinite possible business rule representations resulted from the Authoring Layer. Therefore, it is necessary for the Authoring Layer to generate the Intermediate Representation according to the rule meta data in the Syntax/Vocabulary Model and present the meta data with the format that is agreed upon with the Translation Layer. In this example, HCC conditions and Benefit Limitation conditions are both considered as rule conditions (LHS), while in the Authoring Layer, the distinction is made to offer a more business user-friendly authoring experience. When the time comes for translation, it is the Authoring Layer's responsibility to tell the Translation Layer about such rule meta data via the Intermediate Representation.

IV. STANDARDIZATION OPPORTUNITIES

Syntax Models can be defined using formal grammar languages, such as Backus-Naur-Form (BNF), Extended Backus-Naur-Form (EBNF), Chomsky Normal Form (CNF), etc. In Section III, EBNF is used to specify the History Cross Check (HCC) rule language example. Moreover, language process tools such as yacc/lex, ANTLR are available to generate the Abstract Syntax Tree (AST) which is the core data structure of the Syntax Model.

As for the Vocabulary Model, the core data structure is the vocabulary mapping table as shown in Table I. We believe that it would be worthy of standardization effort being made on the approaches by which this data structure is specified. The argument binding relationship specification needs to be taken into account as well. A point of clarification here is that this is at a different level of abstraction than that of SBVR. While SBVR deals with the specification of vocabulary of a Business Rule, the specification here deals more with a Production Rule that can be executable in a Production (Rule) System.

Standardization work on production rule specifications such as OMG's Production Rule Representation (PRR) [4] and W3C's Rule Interchange Format - Production Rule Dialect (RIF-PRD) [3] address the standardization of the technical production rule that can be adopted by the various rule engine vendors. As discussed in section III-C, we have investigated why it is necessary to have the Intermediate Representation. It can be foreseen that a standardized Intermediate Representation would benefit the framework with more interoperability and the feasibility of using PRR [4], RIF [2] and RIF-PRD [3] for the Intermediate Representation needs investigation. However, there is still an opportu-

nity to address the standardization of BRMS platforms that offer the ability to author a production rule in a domain specific, natural language like fashion. To fulfill that goal there is a compelling need for a standard markup language to specify the mappings between the natural language like domain specific vocabulary and the underlying object specification. This paper addresses the overall blueprint that will unify these different specifications for a holistic BRMS. The details of how this can be addressed concretely will be the subject of a future paper.

Filling these gaps in standardization sets the stage for rule vendors to provide equivalent offerings in the BRMS space which is currently missing. To effectively do that an overall architectural blueprint is required, which is the subject of this paper. Furthermore, even though the framework currently only addresses Production Rules, there exists an opportunity to expand the scope to other flavors of rules such as Event Condition Action (ECA) rules. In addition, the framework can be extended to be a logic specification platform that addresses the authoring of logic by non-technical business community in a procedural, near natural language which can then be translated to an executable procedural language.

V. CONCLUSION AND FUTURE WORK

This framework can conceptually be thought of as an Enterprise Rule Platform. In many ways it is similar to an Enterprise Service Bus and the value it enables for services. The framework decouples the rule management function from the rule execution function to achieve a vendor neutral enterprise rule architecture that is well shielded from potential technology and product road-map changes. In addition, the framework supports the formulation of multiple domain specific user friendly rule languages to articulate production rules using business terminology. In design this is similar to language frameworks such as Java and .NET where the language and platform are decoupled for purposes of interoperability and ubiquity of languages and platforms. In that sense, the notion of an Intermediate Rule Representation is analogous to the Java Byte Code or .NET Intermediate Language (IL). Figure 9 shows a high level depiction of the framework.

Future work on this framework will center around the enrichment of the meta model/data around the Vocabulary Model and Vocabulary Specification Language. Some of the meta-data could include (but not be limited to) plug-in or injection of customized vocabulary edits on arguments, customized display metaphors such as context sensitive domain help and dynamic selection (based on values stored in a database) of enumerated domain values, trace/correlation functionality of vocabulary and rule to higher level rule constructs such as policy and high level rules. There is also a keen interest in exploring SBVR [5] and integrating that as a possibility for authoring high level policies and contractual stipulations. SBVR [5] which stands for Semantics

of Business Vocabulary and Business Rules is an adopted standard of OMG that strives to document Business Rules in a formal naturalized language. SBVR lends itself better for high level Business Rule and Policy capture.

Other future work for this framework would be to capture system rules as well in a declarative fashion. There is increasing evidence within the enterprise where large amounts of system rules are buried in procedural code and which could benefit from externalization.

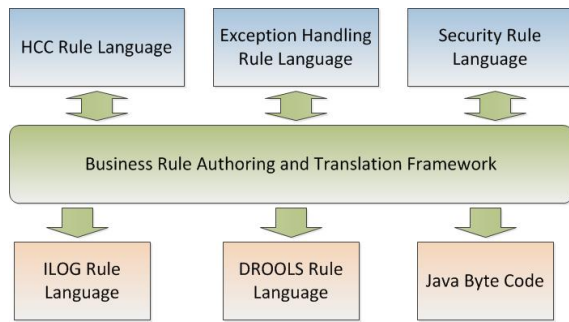


Figure 9. Enterprise Rule Platform

ACKNOWLEDGMENT

The authors would like to sincerely thank the ISS management at Delta Dental of Michigan for their support and funding of the project that is the basis for this paper. The authors would also like to thank the BRIDE Subject Matter Experts and Rule Authors for their continued support and enthusiasm of the BRIDE rules framework.

REFERENCES

- [1] M. Agaram and B. Laird, *A Componentized Architecture for Externalized Business Rules*, Proc. 14th Int. EDOC Conf., pp. 175-183, 2010. IEEE Press.
- [2] RIF. <http://www.w3.org/2005/rules/wiki/Overview>
- [3] RIF-PRD. <http://www.w3.org/2005/rules/wiki/PRD>
- [4] PRR. <http://www.omg.org/spec/PRR/1.0/>
- [5] Semantics of Business Vocabulary and Business Rules (SBVR). <http://www.omg.org/spec/SBVR/1.0/>